

DOCUMENTACIÓN TÉCNICA PROFESIONAL

Análisis de Aplicación .NET | CyberOps Lab

Randy Nin | Matrícula: 2025-0660

Índice

1. Contexto técnico
2. Objetivos
3. Alcance
4. Estructura del binario
 - 4.1 Archivos del paquete
 - 4.2 Por qué el .exe no contiene el código
 - 4.3 Punto de entrada: Program.Main()
5. Análisis estático: clase Form1
 - 5.1 ByteArrayToString
 - 5.2 ConexionErrorMessagebox
 - 5.3 ROT13
 - 5.4 PrintHintMessage
 - 5.5 GenerateSystemFingerPrinting
 - 5.6 ProcessUserInput
 - 5.7 StartHintListener
 - 5.8 Métodos de conversión de formatos
 - 5.9 ConstructString
 - 5.10 StringToB64
 - 5.11 GetRemoteSecret
 - 5.12 Form1_Load
6. Flujo completo del programa
7. Análisis de red y tráfico generado
 - 7.1 Prueba de conectividad
 - 7.2 Puerto del servicio de pistas
 - 7.3 Petición HTTP a GitHub
8. Servicio de pistas: Hint Server
9. Encodings identificados
10. Resolución del reto paso a paso
11. Conclusión
12. Referencias

I. Contexto técnico

El reto consiste en analizar una aplicación de escritorio Windows desarrollada en C# con .NET 8.0 bajo el framework WinForms. El binario no contiene la contraseña hardcodeda en ninguna parte de su código ni sus recursos: toda la información sensible se obtiene dinámicamente en tiempo de ejecución mediante una combinación de técnicas de ofuscación leve, múltiples encodings encadenados y una petición HTTP a un archivo remoto en GitHub.

La aplicación presenta deliberadamente comportamientos ocultos: abre un puerto TCP en localhost al iniciarse, genera un fingerprint único del equipo en ejecución, construye una URL de GitHub mediante piezas ensambladas con cinco técnicas de codificación distintas y valida la contraseña del usuario comparando en Base64 el input recibido contra el contenido hexadecimal del archivo remoto. Para resolver el reto es necesario combinar análisis estático del binario .NET con dnSpy, enumeración de puertos activos con Process Explorer, interacción con el servicio TCP local y decodificación manual de los encodings encontrados.

2. Objetivos

2.1 Objetivo del laboratorio

Demostrar la capacidad de analizar una aplicación .NET desconocida identificando su comportamiento en tiempo de ejecución, los servicios de red que abre, el tráfico que genera hacia el exterior y el mecanismo de validación de la contraseña, resolviendo el reto mediante la decodificación correcta de la cadena hexadecimal obtenida desde GitHub.

2.2 Objetivos específicos

Ejecutar y observar el comportamiento de la aplicación, identificar las comunicaciones de red que genera, descubrir el servicio TCP local y conectarse a él para obtener las pistas, trazar el flujo completo de construcción y validación de la contraseña mediante análisis estático del código fuente decompilado con dnSpy, y decodificar correctamente cada encoding encontrado hasta obtener la contraseña en texto claro.

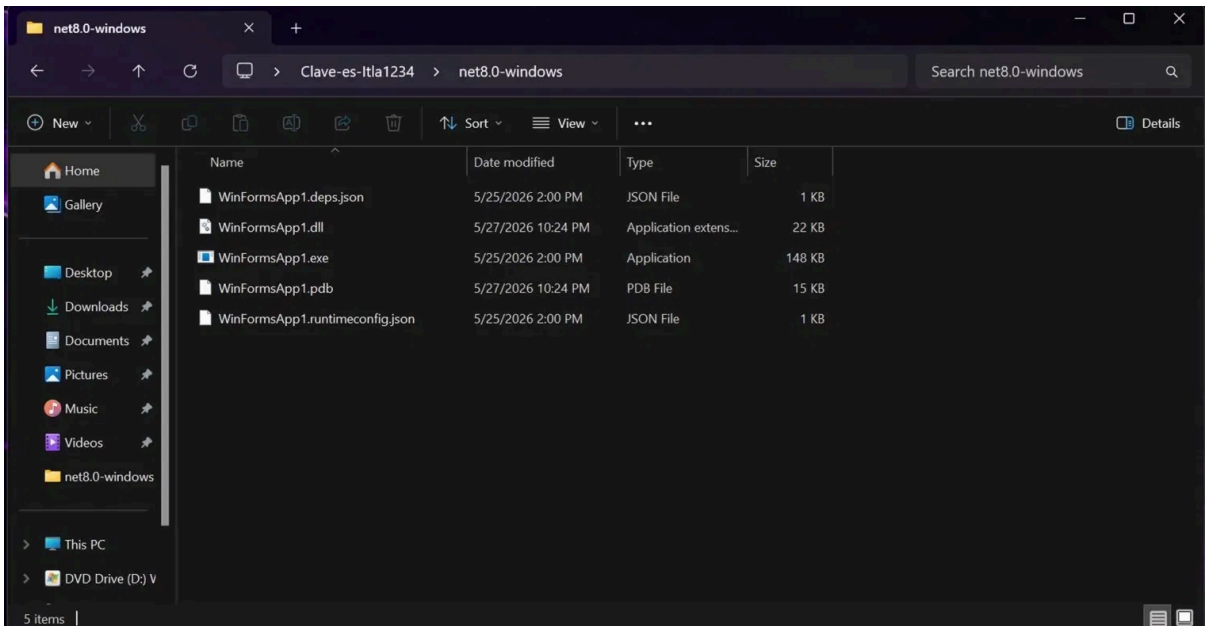
3. Alcance

Esta práctica fue desarrollada con fines exclusivamente académicos dentro del curso CyberOps. Las técnicas de reversing e inspección de tráfico documentadas en este informe aplican únicamente a binarios propios o entregados explícitamente para análisis en entornos de laboratorio controlados.

4. Estructura del binario

4.1 Archivos del paquete

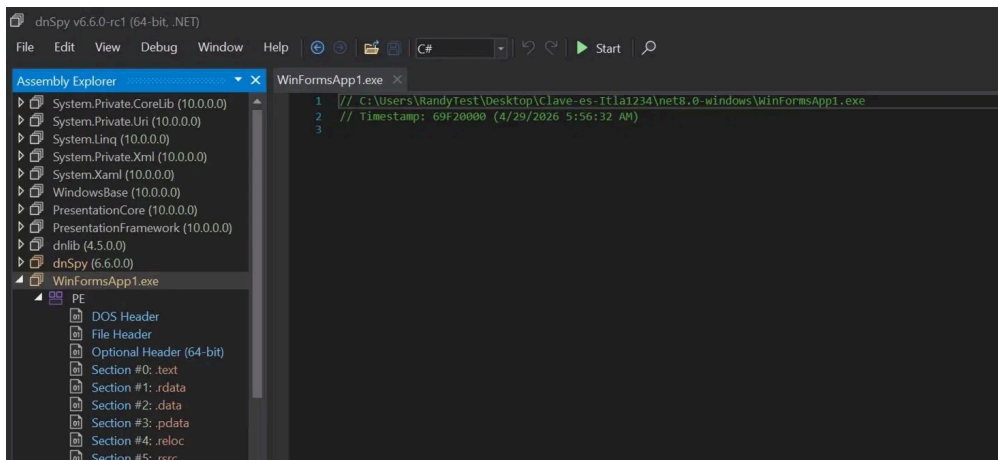
El archivo descargado contiene cinco elementos tras ser descomprimido en la carpeta `net8.0-windows` :



Archivo	Tipo	Función
WinFormsApp1.exe	Ejecutable nativo	Bootstrap de .NET 8.0: lanza el runtime y carga la DLL
WinFormsApp1.dll	Ensamblado .NET gestionado	Contiene todo el código IL de la aplicación
WinFormsApp1.pdb	Archivo de símbolos	Información de depuración (nombres de variables)
WinFormsApp1.deps.json	Manifiesto de dependencias	Lista de paquetes NuGet requeridos
WinFormsApp1.runtimeconfig.json	Configuración del runtime	Especifica que requiere .NET 8.0

4.2 Por qué el .exe no contiene el código

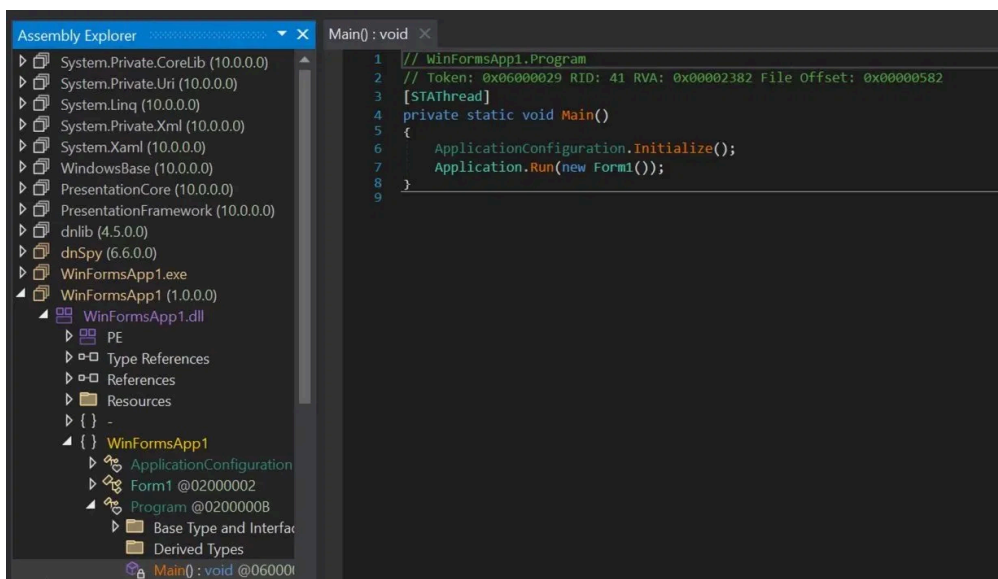
En .NET 8.0 con el modelo de publicación por defecto, el `.exe` es un lanzador nativo que únicamente inicializa el runtime de .NET y delega la ejecución al ensamblado gestionado (`.dll`). Todo el código IL de la aplicación reside en `WinFormsApp1.dll`. Al abrir el `.exe` directamente en dnSpy, el Assembly Explorer únicamente muestra la estructura PE del ejecutable nativo sin ningún namespace ni clase de la aplicación.



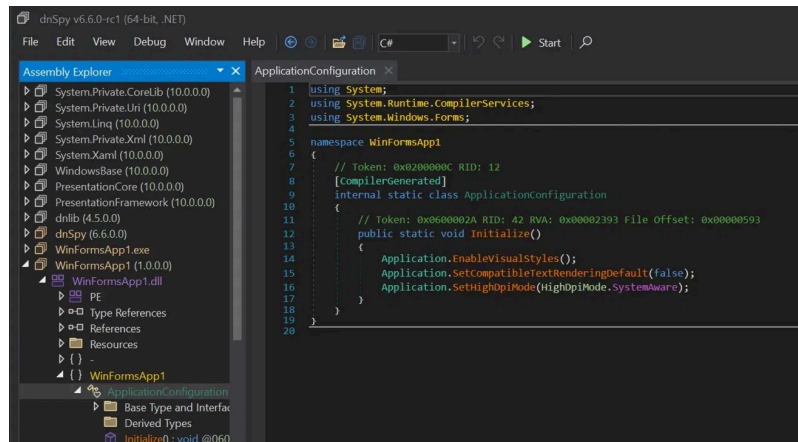
Al cargar `WinFormsApp1.dll` en dnSpy, aparece el namespace `WinFormsApp1` con las clases `ApplicationConfiguration`, `Form1` y `Program`, donde reside todo el código fuente de la aplicación.

4.3 Punto de entrada: Program.Main()

El método `Main` del ensamblado es mínimo: inicializa la configuración visual de la aplicación Windows Forms y lanza la clase `Form1`, que es donde reside toda la lógica del reto.

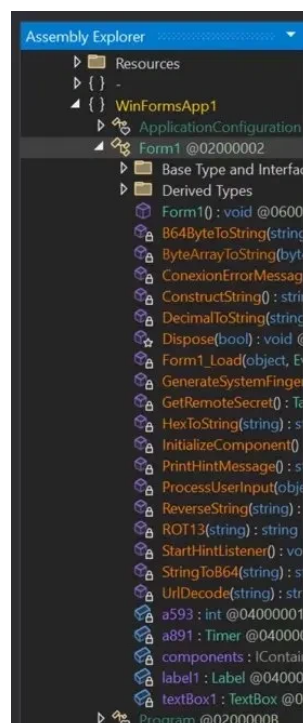


```
[STAThread]
private static void Main()
{
    ApplicationConfiguration.Initialize();
    Application.Run(new Form1());
}
```



5. Análisis estático: clase Form1

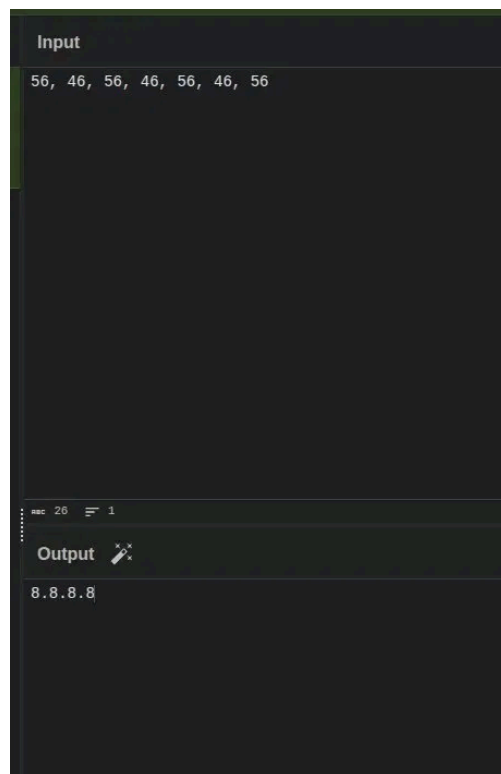
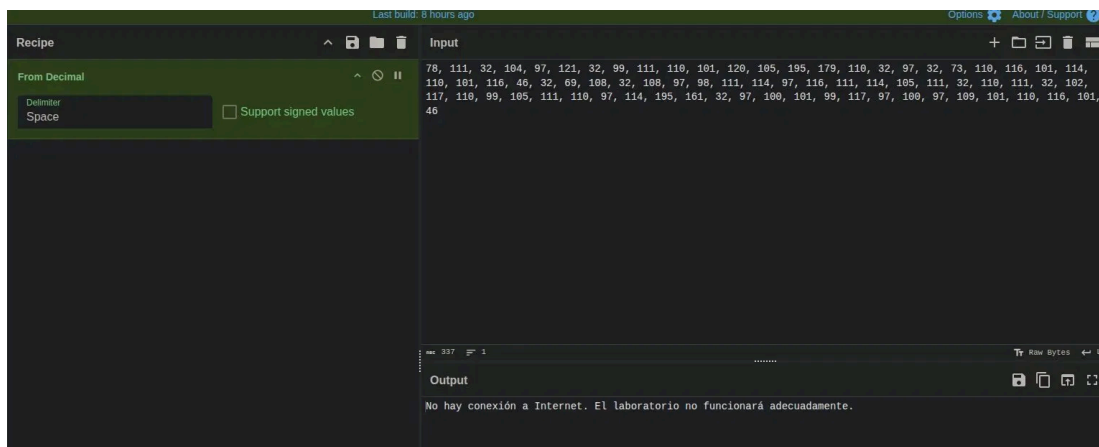
La clase `Form1` hereda de `Form` y contiene todos los métodos del reto. El Assembly Explorer de dnSpy lista cada método en el panel izquierdo, lo que permite identificar la superficie completa del código antes de analizarlo línea por línea.



5.1 ByteArrayToString

Método de utilidad reutilizado en toda la clase. Recibe un array de bytes y lo convierte a su representación en texto UTF-8 mediante `Encoding.UTF8.GetString(b)`. Sirve como capa de ofuscación para todas las cadenas de texto del programa: en lugar de escribir strings literales en el código (fácilmente visibles con un editor hexadecimal o Strings), el desarrollador almacenó cada mensaje, URL y nombre como arrays de valores decimales. Esto incluye el target del ping (8.8.8.8), el mensaje de error de conexión, las etiquetas de los MessageBoxes y las partes de la URL de GitHub.

```
private static string ByteArrayToString(byte[] b)
{
    return Encoding.UTF8.GetString(b);
}
```



5.2 ConexionErrorMessagebox

Este método se registra como handler del tick de un `Timer` de 1 segundo configurado en `Form1_Load`. Cada segundo que la aplicación está en ejecución, intenta enviar un ping ICMP a 8.8.8.8 (obtenida mediante `ByteArrayToString({56, 46, 56, 46, 56, 46, 56})`). Si el ping falla y se lanza una excepción, muestra un `MessageBox` con el mensaje "No hay conexión a Internet. El laboratorio no funcionará adecuadamente.", detiene el timer (`a891.Stop()`) y cierra el formulario.

Este comportamiento implica que la aplicación requiere conectividad a internet activa para funcionar correctamente, ya que la contraseña se obtiene de GitHub en tiempo real.

```
private void ConexionErrorMessagebox(object sender, EventArgs e)
{
    try
    {
        new Ping().Send(Form1.ByteArrayToString(new byte[] { 56, 46, 56, 46, 56, 46, 56 }));
    }
    catch
    {
        MessageBox.Show(Form1.ByteArrayToString(new byte[]
        {
            78, 111, 32, 104, 97, 121, 32, 99, 111, 110,
            101, 120, 105, 195, 179, 110, 32, 97, 32, 73,
            110, 116, 101, 114, 110, 101, 116, 46, 32, 69,
            108, 32, 108, 97, 98, 111, 114, 97, 116, 111,
            114, 105, 111, 32, 110, 111, 32, 102, 117, 110,
            99, 105, 111, 110, 97, 114, 195, 161, 32, 97,
            100, 101, 99, 117, 97, 100, 97, 109, 101, 110,
            116, 101, 46
        })), Form1.ByteArrayToString(new byte[]
        {
            69, 114, 114, 111, 114, 32, 100, 101, 32, 67,
            111, 110, 101, 120, 105, 195, 179, 110
        })), MessageBoxButtons.OK, MessageBoxIcon.Hand);
        this.a891.Stop();
        base.Close();
    }
}
```

5.3 ROT13

Implementa el cifrado César con rotación de 13 posiciones (ROT13). Opera sobre cada carácter de la cadena de entrada: los caracteres alfabéticos en rango `a-z` y `A-Z` son rotados 13 posiciones dentro de su respectivo rango, y cualquier otro carácter (espacios, números, puntuación) se devuelve sin modificar. ROT13 es simétrico: aplicarlo dos veces devuelve el texto original, por lo que codificar y decodificar utilizan exactamente la misma función.


```
private string ROT13(string a106)
{
    return new string(a106.Select<char, char>(delegate(char c)
    {
        if (c >= 'a' && c <= 'z')
        {
            return 'a' + (c - 'a' + '\r') % '\u001a';
        }
        if (c >= 'A' && c <= 'Z')
        {
            return 'A' + (c - 'A' + '\r') % '\u001a';
        }
        return c;
    })).ToArray<char>());
}
```

5.4 PrintHintMessage

Contiene el texto de las pistas del reto, pero no en texto claro. El mensaje se almacena como un string en Base64 que, al ser decodificado, produce el texto con ROT13 aplicado. El método:

1. Decodifica el string Base64 con

```
Encoding.UTF8.GetString(Convert.FromBase64String(text))
```

2. Pasa el resultado por `ROT13()` para obtener el texto legible final
3. Devuelve el texto de las pistas ya en claro

El mensaje final que devuelve (visible al conectarse al hint server) es el listado de 9 pistas que guían la resolución del reto.

```
private string PrintHintMessage()
{
    string text =
        "W1Bsb3JlQmNmIFlubyBVdmFnIEZyZWlyZV0KCjEuIEEd1dmYgY2VidGVueiBnbml4ZiBnYiBUdmdVaG8uCjIuIE  

        d1ciBjbmZmamJlcSB2ZiBhYmcgdW5lcXBicXJxLgozLiBHdXIgSEVZIHZmIG9odnlhIHFoZXZhdCBya3JwaGd2Y  

        mEuCjQuIFliYnggc2JlIHJhcGJxdmF0ZjogVVJLLCBPbmZyNjQsIEVyaXJlZnIgRmdldmF0Zi4KNS4gR3VyIHN2  

        eXIgcGJhZ252YWYgbiBVUksZmdldmF0Lgo2LiBVUksZmdYgYWNlIHJhcGVsY2d2YmEuCjcuIE9uZnI2NCB2ZiB  

        hYmcgcFwZwXjZ3ZiYS4KOC4gSnZlcmZ1bmV4IHZmIGxiaGUgc2V2cmFxLgo5LiBxYUZjbCB2ZiBueWZiIGxiaG  

        Ugc2V2cmFxLgoKVGVjcSB5aHB4IDopCi0gRWJhZgo=";
    string @string = Encoding.UTF8.GetString(Convert.FromBase64String(text));
    return this.ROT13(@string);
}
```


5.5 GenerateSystemFingerprinting

Genera un identificador único del equipo donde se ejecuta la aplicación. El proceso es:

1. Concatena el nombre NetBIOS del equipo con el nombre del usuario activo:

```
Environment.MachineName + ":" + Environment.UserName
```

2. Calcula el hash SHA1 de esa cadena codificada en UTF-8
3. Convierte únicamente los primeros 5 bytes del hash a su representación hexadecimal en mayúsculas con `BitConverter.ToString(..., 0, 5)`
4. Elimina los guiones que `BitConverter.ToString` inserta entre bytes con `.Replace("-", "")`
5. Devuelve los 10 caracteres hexadecimales resultantes

En el equipo del laboratorio, la cadena `RANDY-TESTVM:RandyTest` produjo el fingerprint `FA12B8429A`. Este valor se utiliza en la construcción del Execution ID visible al conectarse al hint server.

```
private string GenerateSystemFingerprint()
{
    string text = Environment.MachineName + ":" + Environment.UserName;
    string text2;
    using (SHA1 sha = SHA1.Create())
    {
        text2 = BitConverter.ToString(sha.ComputeHash(Encoding.UTF8.GetBytes(text)), 0,
            5).Replace("-", "");
    }
    return text2;
}
```

5.6 ProcessUserInput

Este es el núcleo de la validación del reto. Se dispara cada vez que el usuario presiona la tecla Enter dentro del `textBox1`. El flujo completo de validación es el siguiente:

1. Codifica el input del usuario a Base64 con `StringToB64(textBox1.Text)` y lo guarda en `User_Input_in_b64`
2. Llama a `GetRemoteSecret()` de forma asíncrona y obtiene la cadena hexadecimal del archivo de GitHub
3. Convierte el hex a texto legible con `HexToString(text.Trim())`
4. Codifica ese texto a Base64 con `StringToB64(text2)` y lo guarda en `text3`
5. Compara `User_Input_in_b64` con `text3`

Si son iguales, muestra `MessageBox` con título "Éxito" y mensaje "Contraseña correcta". Si no, muestra "Error" con "Contraseña incorrecta". Si `GetRemoteSecret()` falla (sin internet o problema de red), muestra "Error de conexión o validación".

La contraseña nunca se compara en texto claro: ambos lados de la comparación están en Base64. Esto significa que la única forma de extraer la contraseña es obtener el hex de GitHub y decodificarlo manualmente.

```
private async void ProcessUserInput(object sender, KeyPressEventArgs e)
{
    if (e.KeyChar == '\r')
    {
        try
        {
            string User_Input_in_b64 = this.StringToB64(this.textBox1.Text);
            string text = (await this.GetRemoteSecret()).Trim();
            string text2 = this.HexToString(text);
            string text3 = this.StringToB64(text2);
            if (User_Input_in_b64 == text3)
            {
                MessageBox.Show(Form1.ByteArrayToString(new byte[]
                {
                    67, 111, 110, 116, 114, 97, 115, 101, 195, 177,
                    97, 32, 99, 111, 114, 114, 101, 99, 116, 97
                }), Form1.ByteArrayToString(new byte[] { 195, 137, 120, 105, 116, 111 }), MessageBoxButtons.OK, MessageBoxIcon.Asterisk);
            }
            else
            {
                MessageBox.Show(Form1.ByteArrayToString(new byte[]
                {
                    67, 111, 110, 116, 114, 97, 115, 101, 195, 177,
                    97, 32, 105, 110, 99, 111, 114, 114, 101, 99,
                    116, 97
                }), Form1.ByteArrayToString(new byte[] { 69, 114, 114, 111, 114 }), MessageBoxButtons.OK, MessageBoxIcon.Hand);
            }
            User_Input_in_b64 = null;
        }
        catch
        {
            MessageBox.Show(Form1.ByteArrayToString(new byte[]
            {
                69, 114, 114, 111, 114, 32, 100, 101, 32, 99,
                111, 110, 101, 120, 105, 195, 179, 110, 32, 111,
                32, 118, 97, 108, 105, 100, 97, 105, 195,
                179, 110
            }), Form1.ByteArrayToString(new byte[] { 69, 114, 114, 111, 114 }), MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
        }
    }
}
```

5.7 StartHintListener

Inicia un servidor TCP en localhost que escucha conexiones entrantes y entrega las pistas del reto a quien se conecte. El detalle técnico más importante es la gestión del puerto:

```
TcpListener tcpListener = new TcpListener(IPAddress.Loopback, 0);
tcpListener.Start();
this.a593 = ((IPEndPoint)tcpListener.LocalEndpoint).Port;
```

Al pasarle el puerto `0` al constructor de `TcpListener`, el sistema operativo asigna automáticamente cualquier puerto ephemeral disponible. El puerto efectivamente asignado se recupera después del `Start()` desde `LocalEndpoint.Port` y se almacena en el campo `a593`. Esto hace que el puerto sea completamente dinámico y potencialmente diferente en cada ejecución, lo que obliga al analista a descubrirlo activamente en lugar de poder hardcodearlo.

El servidor corre en un hilo daemon separado. Por cada cliente que se conecta, envía:

- La cabecera `== CyberOps Lab Hint Server by Rons ==`
- La línea `Execution ID: <fingerprint>-<puerto>` (por ejemplo: `FA12B8429A-50052`)
- El texto completo de las pistas obtenido de `PrintHintMessage()`
- `Type anything or press ENTER to close.`
- Espera que el cliente envíe una línea y luego cierra la conexión

```
private void StartHintListener()
{
    Task.Run(delegate
    {
        try
        {
            TcpListener tcpListener = new TcpListener(IPAddress.Loopback, 0);
            tcpListener.Start();
            this.a593 = ((IPEndPoint)tcpListener.LocalEndpoint).Port;
            for (;;)
            {
                TcpClient tcpClient = tcpListener.AcceptTcpClient();
                using (NetworkStream stream = tcpClient.GetStream())
                {
                    using (StreamWriter streamWriter = new StreamWriter(stream, Encoding.ASCII))
                    {
                        using (StreamReader streamReader = new StreamReader(stream, Encoding.ASCII))
                        {
                            streamWriter.AutoFlush = true;
                            streamWriter.WriteLine(Format1.ByteArrayToString(new byte[]
                            {
                                61, 61, 32, 67, 121, 98, 101, 114, 79, 112,
                                115, 32, 76, 97, 98, 32, 72, 105, 110, 116,
                                32, 83, 101, 114, 118, 101, 114, 32, 98, 121,
                                32, 82, 111, 110, 115, 32, 61, 61
                            }));
                            streamWriter.WriteLine(Format1.ByteArrayToString(new byte[]
                            {
                                69, 120, 101, 99, 117, 116, 105, 111, 110, 32,
                                73, 68, 58, 32
                            }) + this.GenerateSystemFingerprinting() + Format1.ByteArrayToString(new byte[] { 45 }) + this.a593.ToString());
                            streamWriter.WriteLine();
                            foreach (string text in this.PrintHintMessage().Split("\n", StringSplitOptions.None))
                            {
                                streamWriter.WriteLine(text.TrimEnd());
                                Thread.Sleep(100);
                            }
                            streamWriter.WriteLine();
                            streamWriter.WriteLine(Format1.ByteArrayToString(new byte[]
                            {
                                84, 121, 112, 101, 32, 97, 110, 121, 116, 104,
                                105, 110, 103, 32, 111, 114, 32, 112, 114, 101,
                                115, 115, 32, 69, 78, 84, 69, 82, 32, 116,
                                111, 32, 99, 108, 111, 115, 101, 46
                            }));
                            streamReader.ReadLine();
                        }
                    }
                }
                tcpClient.Close();
            }
        }
        catch
        {
        }
    });
}
```

5.8 Métodos de conversión de formatos

La clase incluye cuatro métodos de conversión reutilizados principalmente por

`ConstructString`:

Método	Entrada	Operación	Ejemplo
<code>HexToString</code>	String hexadecimal	Convierte cada par de caracteres hex a su byte correspondiente y decodifica UTF-8	"30633436" -> "0c46"
<code>B64ByteToString</code>	String en Base64	Decodifica Base64 y convierte los bytes a UTF-8	"MDAwZA==" -> "000d"
<code>DecimalToString</code>	String de decimales separados por -	Convierte cada número decimal a su carácter ASCII correspondiente	"53-52-100-57" -> "54d9"
<code>ReverseString</code>	Cualquier string	Invierte el orden de los caracteres	"4136" -> "6314"
<code>UrlDecode</code>	String URL-encoded	Decodifica secuencias %XX a sus caracteres ASCII equivalentes	"%38%39%35%36" -> "8956"

5.9 ConstructString

Este método construye el hash del commit de GitHub

(`0c46000d54d963148956df879eb518d384c5cf64`) necesario para armar la URL del archivo con la contraseña. El hash está fragmentado en 10 segmentos, cada uno codificado con un método diferente, y se ensamblan secuencialmente en un `StringBuilder`. Esta fragmentación es la principal medida de ofuscación del programa: ningún segmento de la URL aparece completo en el binario.

Segmento	Método	Input	Output
1	<code>HexToString</code>	"30633436"	0c46
2	<code>B64ByteToString</code>	"MDAwZA=="	000d
3	<code>DecimalToString</code>	"53-52-100-57"	54d9
4	<code>ReverseString</code>	"4136"	6314
5	<code>UrlDecode</code>	"%38%39%35%36"	8956
6	<code>HexToString</code>	"64663837"	df87

Segmento	Método	Input	Output
7	B64ByteToString	"0WViNQ=="	9eb5
8	DecimalToString	"49-56-100-51"	18d3
9	ReverseString	"5c48"	84c5
10	HexToString	"63663634"	cf64

Concatenando los 10 segmentos: 0c46 + 000d + 54d9 + 6314 + 8956 + df87 + 9eb5 + 18d3 + 84c5 + cf64 = 0c46000d54d963148956df879eb518d384c5cf64

```
private string ConstructString()
{
    StringBuilder stringBuilder = new StringBuilder();
    stringBuilder.Append(this.HexToString("30633436"));
    stringBuilder.Append(this.B64ByteToString("MDAwZA=="));
    stringBuilder.Append(this.DecimalToString("53-52-100-57"));
    stringBuilder.Append(this.ReverseString("4136"));
    stringBuilder.Append(this.UrlDecode("%38%39%35%36"));
    stringBuilder.Append(this.HexToString("64663837"));
    stringBuilder.Append(this.B64ByteToString("0WViNQ=="));
    stringBuilder.Append(this.DecimalToString("49-56-100-51"));
    stringBuilder.Append(this.ReverseString("5c48"));
    stringBuilder.Append(this.HexToString("63663634"));
    return stringBuilder.ToString();
}
```

Codifica una cadena de texto a Base64 usando

`Convert.ToBase64String(Encoding.UTF8.GetBytes(a106))`. Se utiliza en `ProcessUserInput` para convertir tanto el input del usuario como la contraseña obtenida de GitHub al mismo formato antes de la comparación.

```
private string StringToB64(string a106)
{
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(a106));
}
```

5.II GetRemoteSecret

Realiza la petición HTTP que obtiene la contraseña remota. Construye la URL final combinando tres componentes, todos almacenados como arrays de bytes para evitar su aparición en texto claro:

Componente	Array decimal	Valor resultante
text (base de la URL)	{104, 116, 116, 112, 115, 58, 47, 47, 114, 97...}	https://raw.githubusercontent.com/JonathanERC
text3 (hash del commit)	Resultado de ConstructString()	0c46000d54d963148956df879eb518d384c5cf64
text2 (nombre del archivo)	{114, 111, 110, 115, 95, 115, 114, 46, 116, 120, 116}	rons_sr.txt

La URL final construida es:

```
https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/0c46000d54d963148956df879eb518d384c5cf64/rons_sr.txt
```

La petición se realiza con `HttpClient` usando el User-Agent `CyberOps-Lab` (obtenido de `ByteArrayToString({67, 121, 98, 101, 114, 79, 112, 115, 45, 76, 97, 98})`). El archivo devuelve una cadena hexadecimal que al ser decodificada contiene la contraseña.

```
private async Task<string> GetRemoteSecret()
{
    string text = Form1.ByteArrayToString(new byte[]
    {
        104, 116, 116, 112, 115, 58, 47, 47, 114, 97,
        119, 46, 103, 105, 116, 104, 117, 98, 117, 115,
        101, 114, 99, 111, 110, 116, 101, 110, 116, 46,
        99, 111, 109, 47, 74, 111, 110, 97, 116, 104,
        97, 110, 69, 82, 67, 47, 83, 99, 114, 105,
        112, 116, 115, 80, 79, 67, 47
    });
    string text2 = Form1.ByteArrayToString(new byte[]
    {
        114, 111, 110, 115, 95, 115, 114, 46, 116, 120,
        116
    });
    string text3 = this.ConstructString();
    string text4 = text + text3 + "/" + text2;
    string text5;
    using (HttpClient request = new HttpClient())
    {
        request.DefaultRequestHeaders.UserAgent.ParseAdd(Form1.ByteArrayToString(new byte[]
        {
            67, 121, 98, 101, 114, 79, 112, 115, 45, 76,
            97, 98
        }));
        text5 = await request.GetStringAsync(text4);
    }
    HttpClient request = null;
    return text5;
}
```

5.12 Form1_Load

Es el método que inicializa toda la actividad del programa al cargar el formulario. Ejecuta dos acciones:

1. Crea el `Timer a891` con intervalo de 1000 ms, le asigna `ConexionErrorMessagebox` como handler del tick y lo inicia. Esto activa la verificación de conectividad cada segundo.
2. Llama a `StartHintListener()`, que lanza el servidor TCP de pistas en un hilo separado.

Ambas acciones ocurren de forma transparente para el usuario, que únicamente ve el formulario con el campo de contraseña.

```
private void Form1_Load(object sender, EventArgs e)
{
    this.a891 = new global::System.Windows.Forms.Timer();
    this.a891.Interval = 1000;
    this.a891.Tick += this.ConexionErrorMessagebox;
    this.a891.Start();
    this.StartHintListener();
}
```

6. Flujo completo del programa

El siguiente diagrama describe la secuencia de eventos desde el inicio hasta la validación exitosa:



|

v

Usuario abre Process Explorer

-> identifica WinFormsApp1.exe

-> TCP/IP tab: puerto en LISTENING (50052)

|

v

Usuario se conecta: ncat 127.0.0.1 50052

-> Recibe: Execution ID: FA12B8429A-50052

-> Recibe: 9 pistas del reto (Base64 + ROT13)

|

v

Usuario analiza pistas:

"This program talks to GitHub"

"The password is not hardcoded"

"The file contains a HEX string"

|

v

Análisis estático con dnSpy sobre WinFormsApp1.dll

-> ConstructString() ensambla el hash del commit
usando 5 encodings distintos (HEX, B64, Decimal,
Reverse, URL)

-> GetRemoteSecret() construye la URL y hace GET
[https://raw.githubusercontent.com/JonathanERC/
ScriptsPOC/0c46.../rons_sr.txt](https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/0c46.../rons_sr.txt)

|

v

GitHub devuelve HEX:

4974206e657665722068617070656e73...

|

v

CyberChef: From Hex

-> "It never happens all at once. Its slow.
Its methodical. Its exhausting."

|

v

Usuario introduce la contraseña en la app

-> ProcessUserInput():

```
input_b64 == HexToString(remote_hex) en b64?  
SI -> MessageBox "Contraseña correcta"
```

7. Análisis de red y tráfico generado

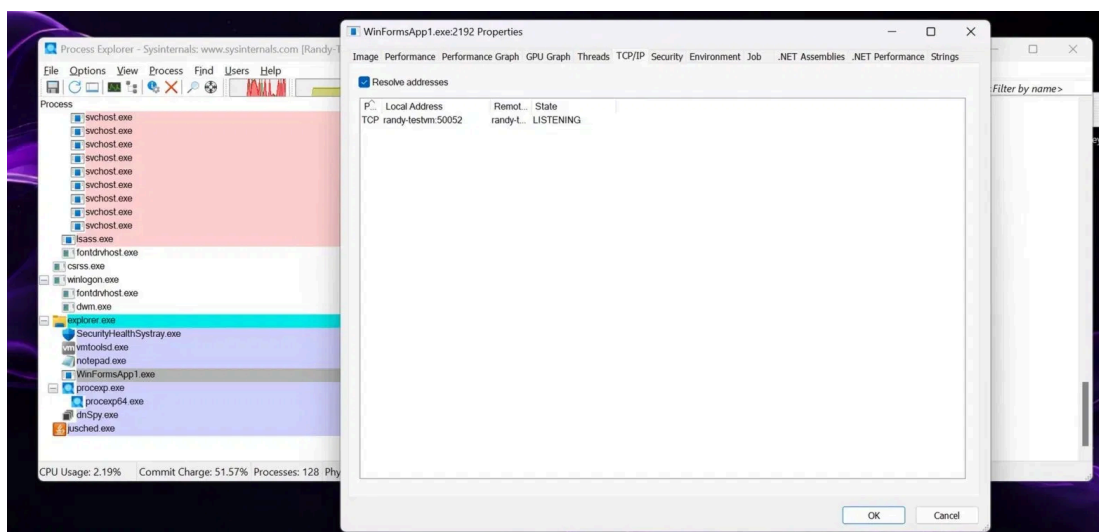
7.1 Prueba de conectividad

La aplicación realiza una prueba de conectividad enviando un ping ICMP al servidor DNS público de Google `8.8.8.8` cada segundo mediante el método `ConexionErrorMessagebox`. La dirección IP no aparece como string literal en el código: se obtiene decodificando el array `{56, 46, 56, 46, 56, 46, 56}` mediante `ByteArrayToString`, que convierte los valores decimales `56='8', 46='.', 56='8', 46='.', 56='8', 46='.', 56='8'` a la cadena `"8.8.8.8"`. Si el ping lanza una excepción, la aplicación muestra el mensaje de error y se cierra automáticamente.

7.2 Puerto del servicio de pistas

Al iniciarse, la aplicación abre un listener TCP en localhost con puerto `0`, lo que delega la asignación del puerto al sistema operativo. El OS elige cualquier puerto ephemeral disponible, haciendo que el número de puerto sea diferente en cada ejecución. El puerto asignado se almacena internamente y se incluye en el Execution ID que el hint server envía al conectarse.

Para descubrirlo, se utiliza Process Explorer de Sysinternals: seleccionando el proceso `WinFormsApp1.exe` y abriendo la pestaña TCP/IP de sus propiedades, se observa el puerto en estado `LISTENING` en `127.0.0.1`.



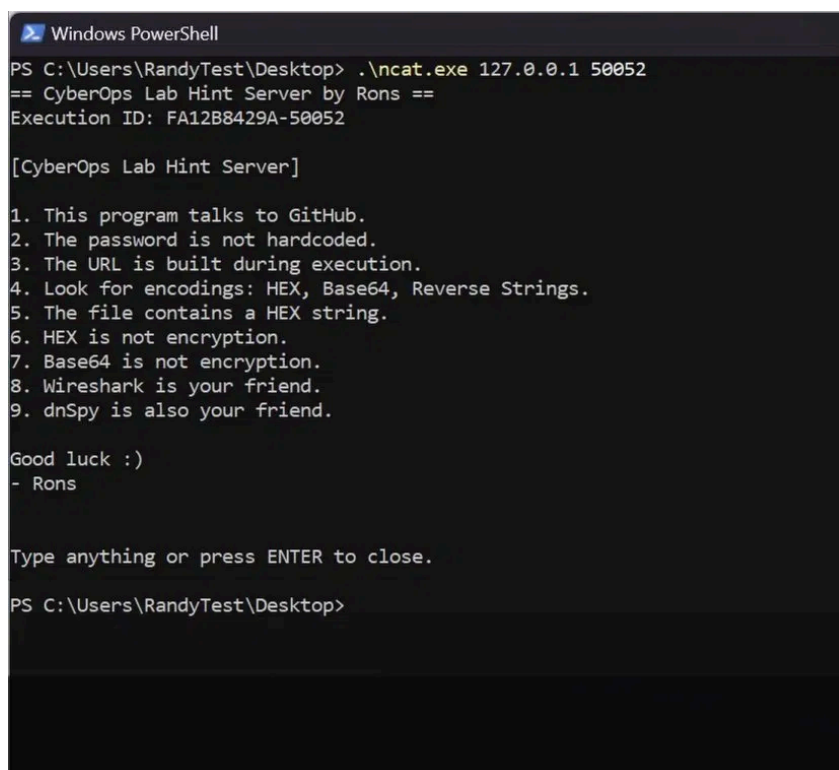
7.3 Petición HTTP a GitHub

El único tráfico de red saliente que genera la aplicación (además del ping ICMP periódico) es una petición HTTP GET al archivo `rons_sr.txt` en GitHub cuando el usuario presiona Enter para validar su contraseña. La petición incluye el User-Agent `CyberOps-Lab` para identificar su origen. Esta petición puede ser capturada con Wireshark filtrando por `http && ip.addr == 185.199.x.x` (IPs de GitHub CDN).

8. Servicio de pistas: Hint Server

Para conectarse al servicio de pistas, se utiliza `ncat` (o cualquier cliente TCP como `telnet` o `nc`) apuntando a `127.0.0.1` y el puerto descubierto en Process Explorer:

```
.\ncat.exe 127.0.0.1 50052
```



```
Windows PowerShell
PS C:\Users\RandyTest\Desktop> .\ncat.exe 127.0.0.1 50052
== CyberOps Lab Hint Server ==
Execution ID: FA12B8429A-50052

[CyberOps Lab Hint Server]

1. This program talks to GitHub.
2. The password is not hardcoded.
3. The URL is built during execution.
4. Look for encodings: HEX, Base64, Reverse Strings.
5. The file contains a HEX string.
6. HEX is not encryption.
7. Base64 is not encryption.
8. Wireshark is your friend.
9. dnSpy is also your friend.

Good luck :)
- Rons

Type anything or press ENTER to close.

PS C:\Users\RandyTest\Desktop>
```

El texto recibido es el resultado de decodificar Base64 y aplicar ROT13 al string almacenado en `PrintHintMessage`. El Execution ID `FA12B8429A-50052` expone tanto el fingerprint del equipo (SHA1 de "RANDY-TESTVM:RandyTest" truncado a 5 bytes) como el puerto asignado por el OS en esta ejecución.

9. Encodings identificados

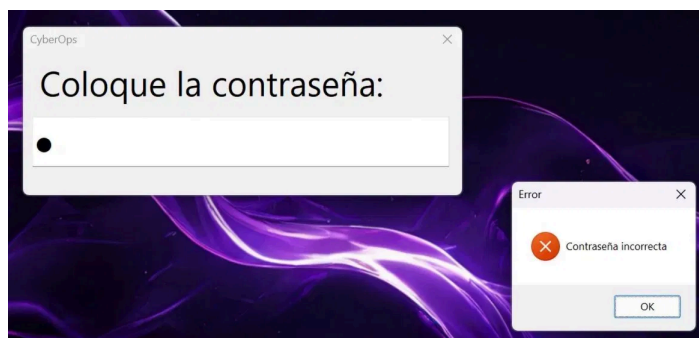
La aplicación utiliza seis técnicas de codificación distintas a lo largo de su código. Ninguna de ellas es cifrado: todas son transformaciones reversibles sin clave.

Encoding	Dónde se utiliza	Herramienta para decodificar
Decimal a UTF-8	Todas las strings de texto del programa (<code>ByteArrayToString</code>): mensajes, URLs, User-Agent	CyberChef: From Decimal
HEX	Contenido de <code>rons_sr.txt</code> en GitHub (la contraseña), y segmentos 1, 6 y 10 de <code>ConstructString</code>	CyberChef: From Hex
Base64	Texto del hint message en <code>PrintHintMessage</code> , y segmentos 2 y 7 de <code>ConstructString</code>	CyberChef: From Base64
ROT13 (cifrado César)	Texto del hint message en <code>PrintHintMessage</code> (aplicado después de decodificar Base64)	CyberChef: ROT13
Reverse String	Segmentos 4 y 9 de <code>ConstructString</code>	CyberChef: Reverse, o manualmente
URL Encoding (%XX)	Segmento 5 de <code>ConstructString</code>	CyberChef: URL Decode

10. Resolución del reto paso a paso

10.1 Ejecutar la aplicación y observar el comportamiento

Al ejecutar `WinFormsApp1.exe`, aparece el formulario con el campo "Coloque la contraseña:". Al introducir cualquier texto incorrecto y presionar Enter, se muestra el MessageBox "Contraseña incorrecta", confirmando que la validación está activa.



Con Process Explorer abierto y el proceso `WinFormsApp1.exe` seleccionado, se abre la ventana de propiedades y se navega a la pestaña TCP/IP. Se observa un socket TCP en estado `LISTENING` sobre el puerto 50052 en `127.0.0.1`.

10.3 Conectarse al hint server

Desde PowerShell, se ejecuta `.\ncat.exe 127.0.0.1 50052`. El servidor responde con el Execution ID y las 9 pistas. Las más relevantes: "This program talks to GitHub", "The password is not hardcoded", "The URL is built during execution" y "The file contains a HEX string".

10.4 Análisis estático con dnSpy

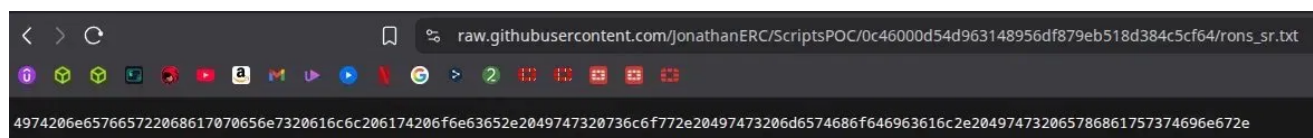
Se carga `WinFormsApp1.dll` en dnSpy. Navegando por `Form1`, se identifica `GetRemoteSecret()` que construye la URL de GitHub mediante `ConstructString()`. Trazando `ConstructString()`, se obtiene el hash del commit ensamblado desde 10 fragmentos con 5 encodings distintos:

```
0c46 + 000d + 54d9 + 6314 + 8956 + df87 + 9eb5 + 18d3 + 84c5 + cf64  
= 0c46000d54d963148956df879eb518d384c5cf64
```

10.5 Acceder al archivo de GitHub y decodificar la contraseña

Se accede directamente en el navegador a la URL construida:

```
https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/0c46000d54d963148956df879eb518d384c5cf64/rons_sr.txt
```



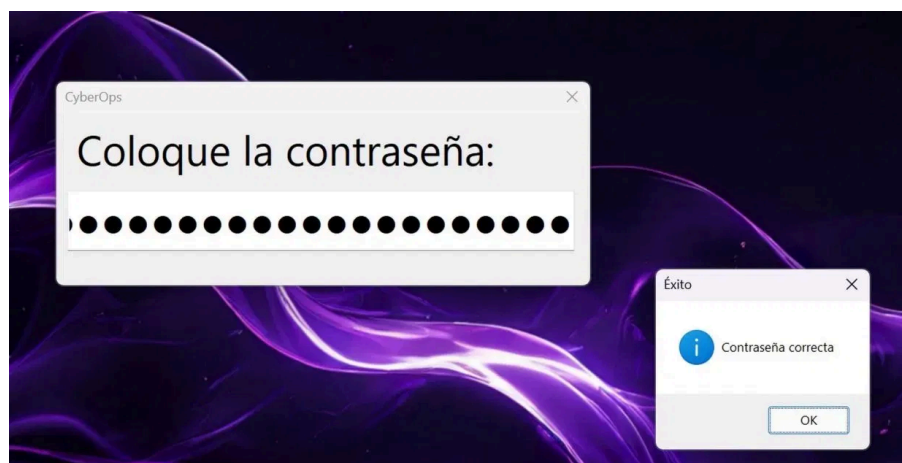
El contenido hexadecimal se pega en CyberChef con la operación `From Hex`:

```
Input                                                                    total:
loaded:
4974206e657665722068617070656e7320616c6c206174206f6e63652e2049747320736c6f772e20497473206
d6574686f646963616c2e204974732065786861757374696e672e

rec: 142  1
Output
It never happens all at once. Its slow. Its methodical. Its exhausting.
```

10.6 Ingresar la contraseña y confirmar la solución

La cadena obtenida `It never happens all at once. Its slow. Its methodical. Its exhausting.` se introduce en el formulario y se presiona Enter.



II. Respuestas a las preguntas del reporte

¿Qué puertos utiliza el programa?

El programa utiliza un único puerto TCP en `127.0.0.1` con modo `LISTENING`, asignado dinámicamente por el sistema operativo al pasarle el valor `0` al constructor de `TcpListener`. El puerto es ephemeral y varía en cada ejecución. En el entorno del laboratorio se asignó el puerto `50052`. No abre ningún puerto hacia el exterior ni hacia otras interfaces de red.

¿Cómo realiza el programa la prueba de conectividad?

Mediante un `Timer` con intervalo de 1000 milisegundos configurado en `Form1_Load`, el método `ConexionErrorMessagebox` se ejecuta una vez por segundo. En cada tick envía un ping ICMP a `8.8.8.8` (DNS público de Google) usando `new Ping().Send(...)`. Si el ping lanza una excepción por falta de conectividad, la aplicación muestra el error de conexión, detiene el timer y cierra el formulario.

Enumera los encodings encontrados y dónde fueron utilizados.

Decimal a UTF-8 en todas las strings del programa a través de `ByteArrayToString`. HEX en el contenido del archivo `rons_sr.txt` (la contraseña) y en tres segmentos de `ConstructString`. Base64 en el texto del hint message dentro de `PrintHintMessage` y en dos segmentos de `ConstructString`. ROT13 en el texto del hint message (aplicado sobre el resultado de decodificar Base64). Reverse String en dos segmentos de `ConstructString`. URL Encoding en un segmento de `ConstructString`.

¿Qué tráfico genera el programa?

Dos tipos: paquetes ICMP Echo Request a `8.8.8.8` cada segundo mientras la aplicación esté abierta (prueba de conectividad), y una petición HTTP GET al archivo `rons_sr.txt` en GitHub con User-Agent `CyberOps-Lab` únicamente cuando el usuario presiona Enter para validar su contraseña.

¿Cómo se conectó al servicio de pistas?

Se identificó el puerto TCP en escucha usando la pestaña TCP/IP de las propiedades del proceso en Process Explorer. Con el puerto identificado (50052 en esta ejecución), se ejecutó `.\ncat.exe 127.0.0.1 50052` desde PowerShell. El hint server respondió automáticamente con el Execution ID y el listado de pistas.

12. Conclusión

El laboratorio demuestra cómo una aplicación puede ocultar de forma efectiva su lógica de validación sin recurrir a cifrado real, utilizando únicamente encodings reversibles encadenados y ofuscación por fragmentación. La contraseña nunca aparece en texto claro dentro del binario: la URL de GitHub está fragmentada en 10 piezas cada una codificada de forma diferente, las strings internas son arrays decimales, y el texto de las pistas está en Base64 más ROT13. A pesar de esto, el análisis estático con dnSpy permite reconstruir todo el flujo en poco tiempo, y las pistas del hint server orientan cada paso del proceso.

El conjunto de técnicas aplicadas: enumeración de servicios con Process Explorer, conexión al servidor local con ncat, análisis estático con dnSpy y decodificación manual con CyberChef, representa el flujo básico de análisis de una aplicación desconocida con comportamiento de red, que es exactamente el objetivo del laboratorio.

13. Referencias

- Repositorio: <https://github.com/RandyNin/REVERSING-ANALYSIS-.NET-APP>
- Video demostrativo: <https://www.youtube.com/watch?v=qKkxRocooWc>
- Archivo remoto secreto:
https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/oc4600od54d963148956df879eb518d384c5cf64/rons_sr.txt
- Herramientas: dnSpy v6.6.0-rc1, Process Explorer (Sysinternals), CyberChef, ncat

Randy Nin / Matrícula 2025-0660